

OBJECT ORIENTED PROGRAMMING (CDT2001)

**B.TECH. SEM II
UNIT I**

Course Objectives

1. **CO1:** To make students understand Fundamental features of an object oriented language like Java: object classes and interfaces, exceptions and libraries of object collections
2. **CO2:** Introduce students with fundamental concepts like exception handling, generics, collection classes and streams.

UNIT I

- Features of Object-Oriented Programming languages, Abstraction, Encapsulation, Inheritance, polymorphism and late binding. Programming paradigms, Bytecode, JDK, JRE, JVM. Concept of a class and object, ways of representing objects, access control of members of a class, instantiating a class, constructor

UNIT II

- Concept of overloading: Constructor Overloading, Function Overloading. Arrays and Array of objects, Wrapper classes (Integer, Double etc.), String Class, creating packages, importing packages. Lambda Expressions Introduction, Block, Passing Lambda expression as Argument

UNIT III

- Concept of inheritance, methods of derivation, use of super keyword and final keyword in inheritance, run time polymorphism, abstract classes and methods, Interface, implementation of interface, static and non-static members

UNIT IV

- Exceptions, types of exception, use of try catch block, handling multiple exceptions, using finally, throw and throws clause, user defined exceptions, Introduction to streams, byte streams, character streams, file handling in Java, Serialization.

- **UNIT V**

Generics, generic class with two type parameter, bounded generics. Collection classes: ArrayList, LinkedList, TreeSet, HashMap, Iterator, ListIterator, Comparator, Comparable

- **UNIT VI**

I: Introduction to Design Patterns, Need of Design Pattern, Classification of Design Patterns, Role of Design Pattern in Software design, Creational Patterns, Structural Design Patterns and Behavioral Patterns

Text Books:

1. Herbert Schildt; JAVA The Complete Reference; Ninth Edition, Tata McGraw- Hill Publishing Company Limited.
2. Design Patterns By Erich Gamma, Pearson Education.

Reference Books:

1. Paul Deitel, Harvey Deitel, Java 9 for Programmers, Pearson
2. Herbert Schildt and Dale Skrien ; Java Fundamentals A Comprehensive Introduction; Tata McGraw- Hill Education Private Ltd 2013

- On successful completion of the course, students will be able to demonstrate
 1. Understand the object-oriented programming features, classes, objects and methods.
 2. Develop efficient programs by implementing the concept of Inheritance, polymorphism exception handling.
 3. Use the concept of generics, collections, streams to develop solution to the given problem.
 4. Analyze characteristics and need of design pattern in software design process

Teaching Scheme

Lecture	3 per week
Tutorial	0
Practical	1 per week
Total Credits	3 (T) 1(P)

Evaluation Scheme

- Internal – 40 marks
- ESE – 60 marks
- Total – 100 marks

1. Features of Object Oriented Programming languages
2. Abstraction
3. Encapsulation
4. Inheritance
5. Polymorphism
6. Late and Early binding
7. Programming paradigms- Bytecode
8. JDK, JRE, JVM
9. Concept of a class and object,
10. Ways of representing objects
11. Access control of members of a class
12. Instantiating a class
13. Constructor

Background and Basics

History of Java

- Designed by James Gosling
- 1992 Oak for electronic devices
- 1995 Official release as Java
- 1999 split into J2SE and J2EE
- 2000 J2ME..

- Java is object-oriented
- OO Programming has been the dominant design approach for large systems
- OO Systems remain testable and enhanceable even as they grow into huge applications with millions of lines of code
- The classes, objects, system states and behaviour in well-designed OO systems are easy to map conceptually to their counterparts in the real world

- Memory Management
 - Unlike C and C++ Java provides automatic memory management
 - In languages that don't provide automatic memory management, keeping track of memory through pointers is quite complex
 - Tracking down bugs related to memory management (often called memory leaks) is a common, error-prone, and time-consuming process
 - In languages that don't provide automatic memory management, keeping track of memory through pointers is quite complex
- Huge Library
 - Java has an enormous library of prewritten, well-tested, and well-supported code
 - This code is easy to include in your Java applications and is well documented via the Java API

- Secure by design
 - When compiled Java code is executed, it runs inside the Java Virtual Machine (JVM)
 - The JVM provides a secure “sandbox” for your Java code to run in
 - The JVM makes sure that nefarious programmers cannot write Java code that will cause trouble on other people’s machines when it runs
 - Write once, run anywhere (cross platform execution)
 - The Java code you write can run on many platforms, ranging from tiny Internet-of-Things (IoT) devices, to phones, to laptop computers, to large servers
- Another common phrase for this ability to run on many devices is cross-platform

- Strongly typed
 - Requires the programmer to explicitly declare the types of the data and objects being used in the program
 - Strong typing allows the Java compiler to catch many potential programming errors before your code even compiles
 - More reliable, testable, production-quality code
- Distributed Computing
 - Another way to tackle big programming problems is to distribute the workload across many machines
 - The Java API provides several ways to simplify tasks related to distributed computing
 - One such example is serialization, a process in which a Java object is converted to a portable form
 - Serialized objects can be sent to other machines, deserialized, and then used as a normal Java object

- Multithreaded
 - Java provides built-in language features and APIs that allow programs to use multiple cores at the same time
 - As systems grow to handle more computationally intensive problems and larger data sets, the ability to use all of a computer's core processors becomes essential
 - Multithreaded programming is never simple, but Java provides a rich toolkit to make it as easy as possible

- 1.0 (1995)
- JDK 1.1 (1996)
- J2SE v 1.2 (a.k.a JDK 1.2, 1998)
- J2SE v 1.3 (a.k.a JDK 1.3, 2000)
- J2SE v 1.4 (a.k.a JDK 1.4, 2002)
- J2SE 5.0 (2004)
- JDK Java SE 6 (2006)
- Java SE 7 (2011)
- Java SE 8 (2014)
- Java SE 9 (2016)
- JAVA SE 10
- JAVA SE 11
- JAVA SE 12
- JAVA SE 13
- JAVA SE 14
- ..
- JAVA 20

- Java Standard Edition (J2SE)
 - J2SE can be used to develop client-side standalone applications or applets.
- Java Enterprise Edition (J2EE)
 - J2EE can be used to develop server-side applications such as Java servlets and Java Server Pages.
- Java Micro Edition (J2ME)
 - J2ME can be used to develop applications for mobile devices such as cell phones.

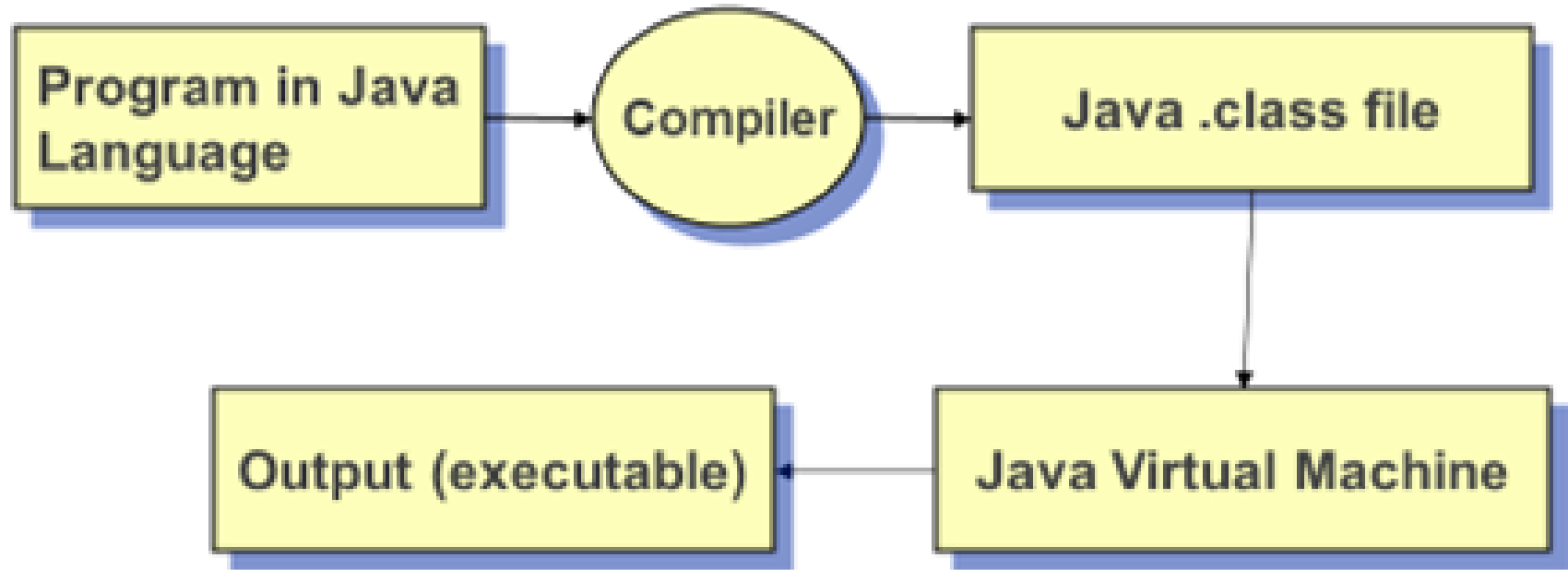
- **JRE (Java Runtime Environment)**
 - Libraries, JVM and other components to run Java application
 - The JRE does not contain tools and utilities such as compilers or debuggers for developing applications
- **Java Development Kit (JDK)**
 - The JDK is a superset of the JRE, and contains everything that is in the JRE, plus tools such as the compilers and debuggers necessary for developing applets and applications

- **JIT (Just In Time Compiler)**
- In JIT not all the code is converted into machine code
- First a part of the code that is necessary will be converted into machine code then if a method or functionality called is not in machine then that will be turned into machine code
- This reduces burden on the CPU

- **OpenJDK**
- Free and Open source version of JDK

- **JAR** stands for Java ARchive
- It's a file format based on the popular ZIP file format and is used for aggregating many files into one
- It's a cross platform format
- Can handle audio, image files and class files

Execution Process



Java Virtual Machine (JVM)

JVM can be meant as a

An **abstract computer architecture**

Software on top of a real hardware

Specification and a Concrete implementation

The world can interact with JVM only through .class file which is platform independent

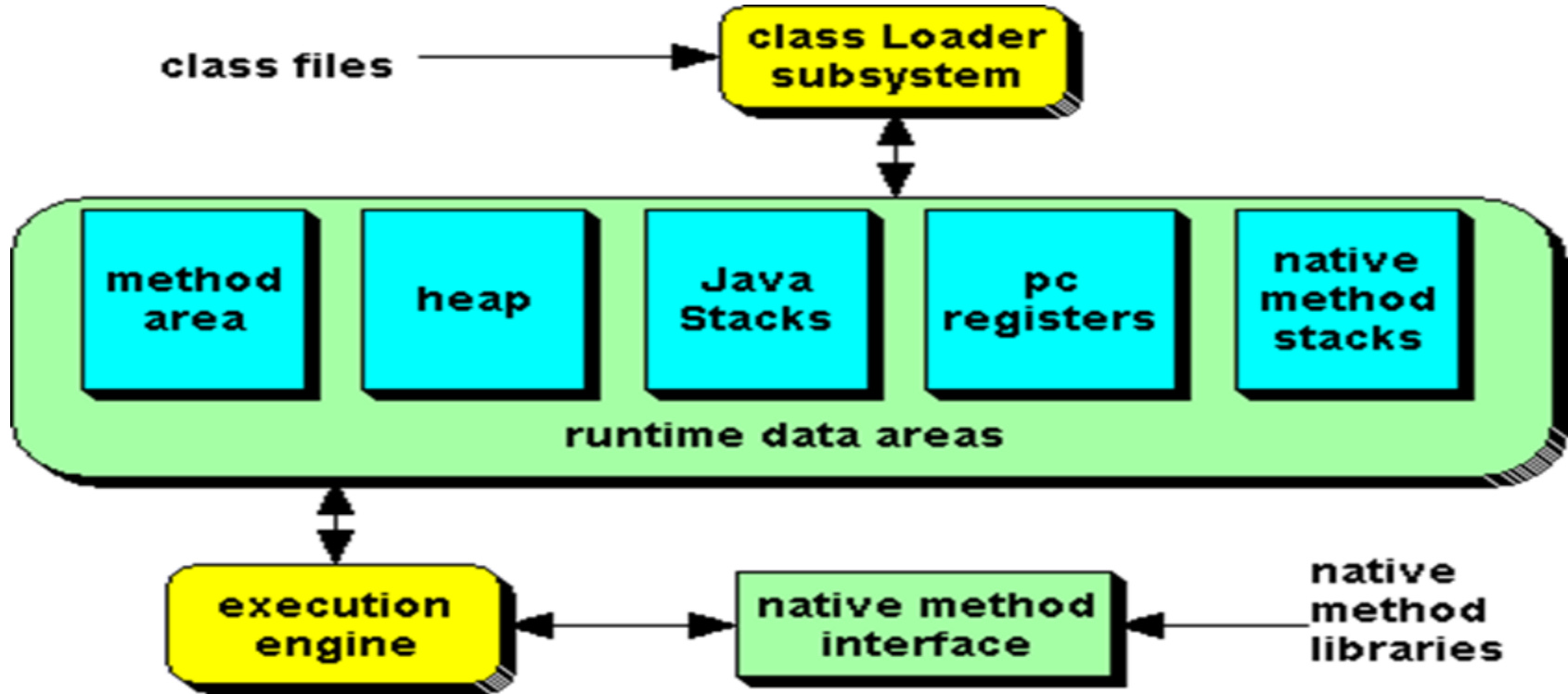
The JVM acts as a **mini OS** providing all resources for executing java program

JVM is platform dependent

A .class file can be executed on any platform provided JVM for that platform is installed. So its

“Write once Run Anywhere”

Basic JVM components & Memory Areas



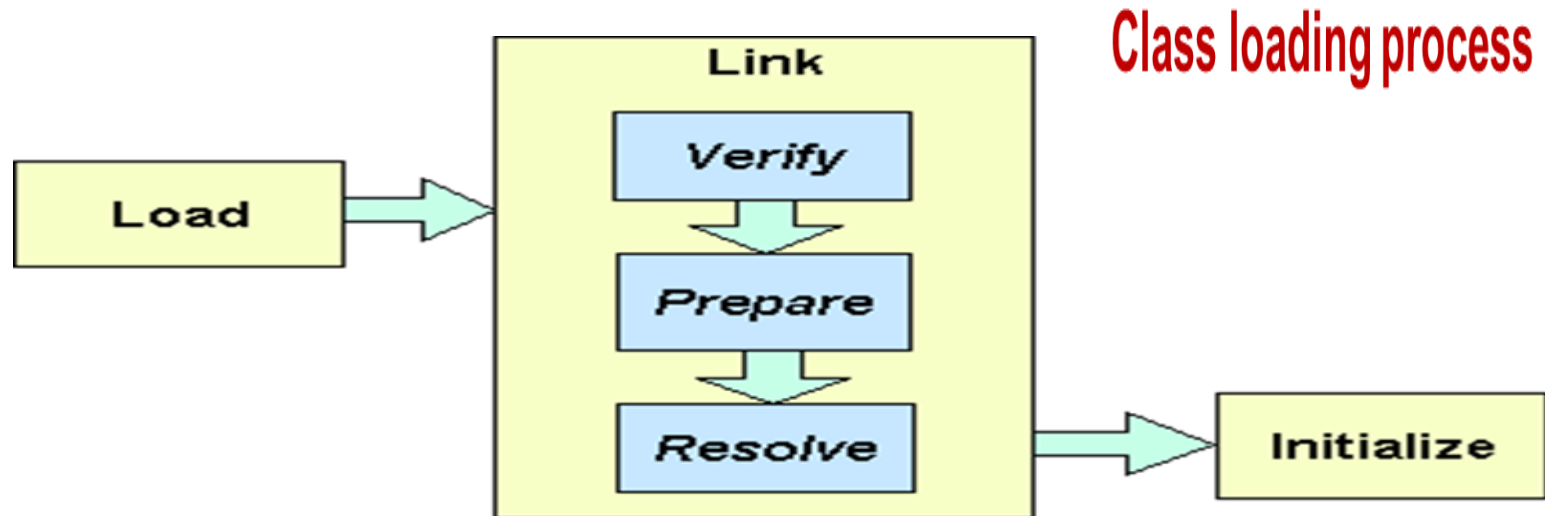
How JVM works?

- Each instance of the JVM has **one method area, one heap, and one or more stacks** - one for each thread
- When JVM loads a class file, it puts its information in the **method area**
- As the program runs, all objects instantiated are stored in the **heap**
- The **stack** area is used to store **activation records** as a program runs

How JVM starts? Contd..

JVM starts by **loading** a particular class and then **invoking** **main()** method in that class

The steps involved are



Java Basics

- Legal identifiers must be composed of only Unicode characters, numbers, currency symbols, and connecting characters (such as underscores)
- Here are the rules you do need to know:
- Identifiers must start with a letter, a currency character (\$), or a connecting character such as the underscore (_).
- Identifiers cannot start with a digit!
- After the first character, identifiers can contain any combination of letters, currency characters, connecting characters, or numbers
- In practice, there is no limit to the number of characters an identifier can contain
- You can't use a Java keyword as an identifier. Table 1-1 lists all the Java keywords
- Identifiers in Java are case sensitive; foo and FOO are two different identifiers.

List of Java Keywords

abstract	boolean	break	byte	case	catch
char	class	const	continue	default	do
double	else	extends	final	finally	float
for	goto	if.	implements	import	instanceof
int	interface	long	native	new	package
private	protected	public	return	short	static
strictfp	super	switch	synchronized	this	throw
throws	transient	try	void	volatile	while
assert	enum				

Examples of identifiers

Examples of legal and illegal identifiers follow. First some legal identifiers:

```
int _a;  
int $c;  
int _____2_w;  
int _$;  
int this_is_a_very_detailed_name_for_an_identifier;
```

The following are illegal (it's your job to recognize why):

```
int :b;  
int -d;  
int e#;  
int .f;  
int 7g;
```

Oracle's Java Code Conventions

- Oracle estimates that over the **lifetime** of a standard piece of code, **20 percent** of the effort will go into the original **creation** and testing of the code, and **80 percent** of the effort will go into the subsequent **maintenance** and **enhancement** of the code.
- **Agreeing** on, and **coding** to, a **set of code standards** helps to **reduce** the **effort** involved in **testing, maintaining, and enhancing** any piece of code
- “Java Code Conventions,” - you can find if you start at java.oracle.com

- **Classes and interfaces** The first letter should be capitalized, and if several words are linked together to form the name, the first letter of the inner words should be uppercase (a format that's sometimes called “**CamelCase**”/“**Pascal Case**”). For classes, the names should typically be **nouns**.
- Here are some examples:
 - Dog
 - Account
 - PrintWriter
 - For **interfaces**, the names should typically be **adjectives**, like these:
 - Runnable
 - Serializable

Methods:

- The **first letter should be lowercase**, and then normal **CamelCase rules** should be used
- In addition, the names should typically be **verb-noun** pairs. For example:
- getBalance
- doCalculation
- setCustomerName

Variables :

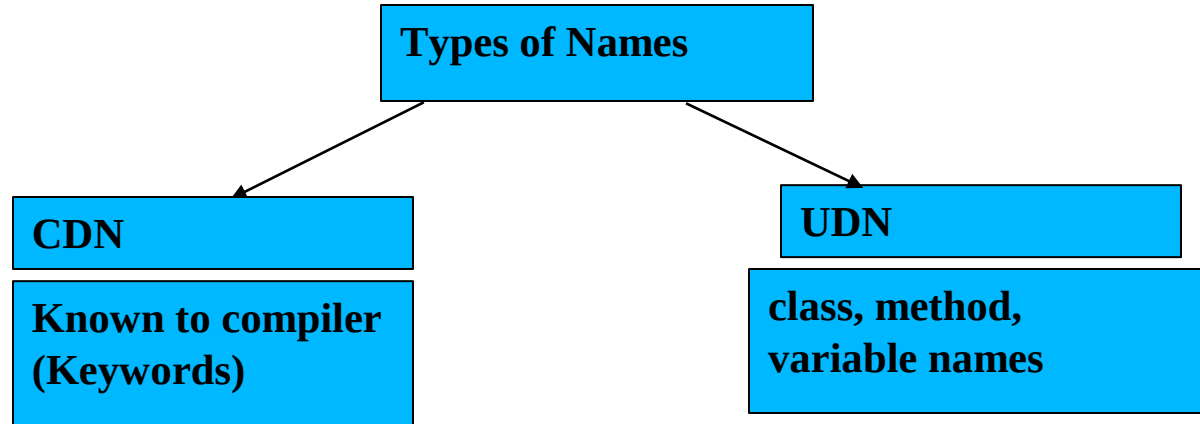
- Like methods, the **CamelCase** format should be used, but **starting with a lowercase letter**.
- Oracle recommends **short, meaningful names**, which sounds good to us.
- Some examples:
 - buttonWidth
 - accountBalance
 - myString

Constants

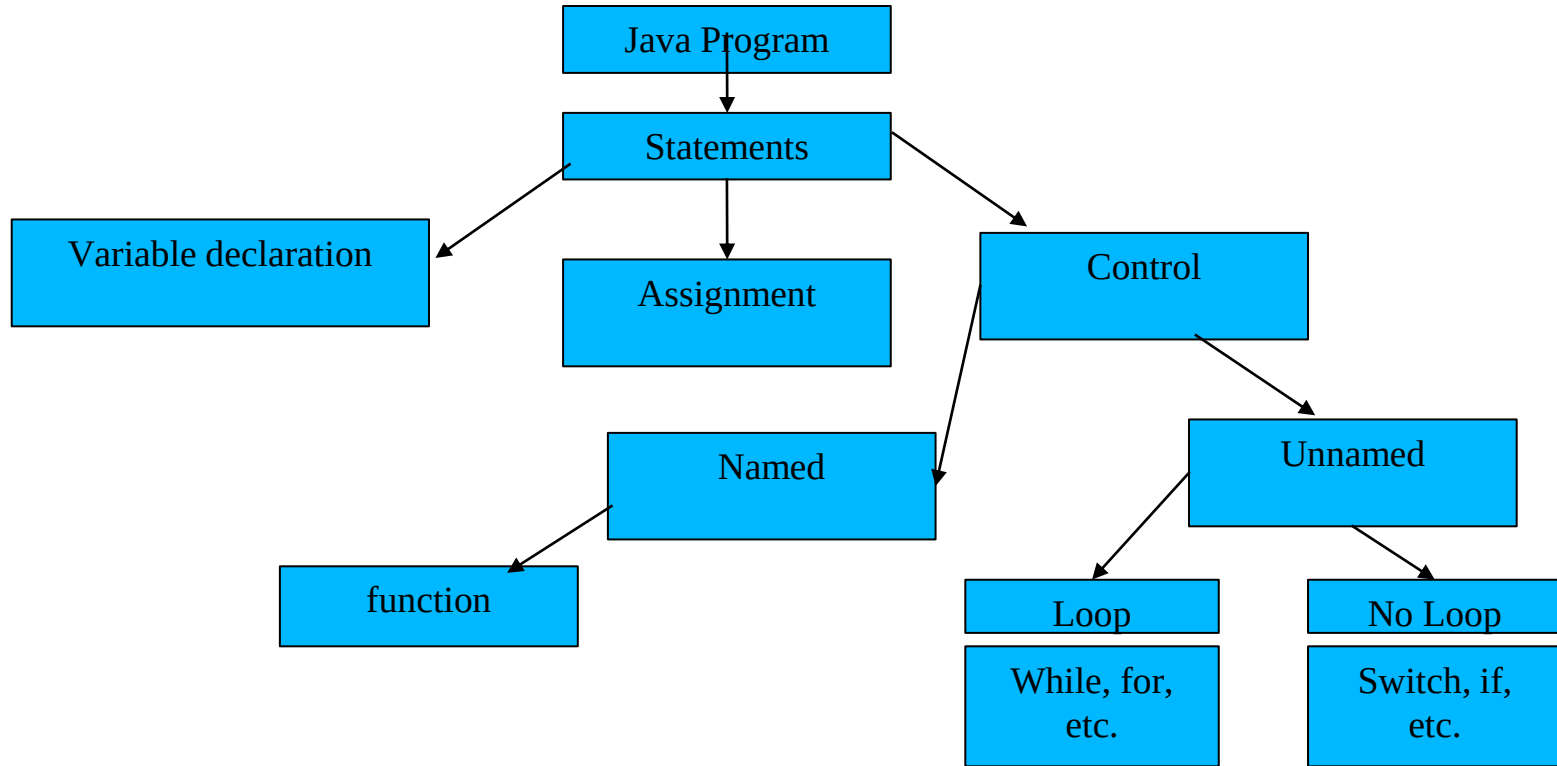
- Java constants are created by marking variables **static and final**
- They should be named using **uppercase letters** with **underscore** characters as separators:
- MIN_HEIGHT
- MAX_DISCOUNT

Getting started.. Java Naming Conventions

- Java is case sensitive



Structure of Java Program



Structure of Java Program

- When you write code in Java,
- You're writing **classes or interfaces**
- **Within** those classes, are **variables and methods**
- (plus a few other things)

Source File Declaration Rules

- There can be **only one public class per source code file**
- **Comments** can appear at the beginning or end of any line in the source code file; they are independent of any of the positioning rules discussed here.
- If there is a **public** class in a file, the **name** of the **file** must match the name of the **public class**. For example, a class declared as `public class Dog { }` must be in a source code file named `Dog.java`.
- If the class is part of a package, the **package statement must be the first line** in the source code file, before any import statements that may be present.
- If there are **import** statements, they must go between the package statement (if there is one) and the class declaration.
- If there isn't a package statement, then the import statement(s) must be the first line(s) in the source code file.
- If there are no package or import statements, the class declaration must be the first line in the source code file.

Source File Declaration Rules

- **import and package statements apply to all classes within a source code file.** In other words, there's no way to declare multiple classes in a file and have them in different packages or use different imports
- **A file can have more than one non-public class**
- **Files with no public classes can have a name that does not match any of the classes in the file**

Compiling Java Program

- Compile the program with javac
 - `javac [options] [source files]`
 - `javac Abc.java` (javac is java compiler to create .class file)
 - This step creates a file called `Abc.class`
 - `javac -help`
 - `javac -version Foo.java Bar.java`

- Launching application with Java
 - `java [options] class [args]`
 - `java Abc` (here java is the JVM loader which loads JVM with the class file supplied as argument)
- The [options] and [args] parts of the java command are optional, and they can both have multiple values
- You must specify exactly one class file to execute, and the java command assumes you're talking about a .class file, so you don't specify the .class extension on the command line
- `java -showversion MyClass x 1`
- This command can be interpreted as "Show me the version of the JVM being used, and then launch the file named MyClass.class and send it two String arguments whose values are x and 1."

Example

```
public class MyClass {  
    public static void main(String[] args) {  
        System.out.println(args[0] + " " + args[1]);  
    }  
}
```

It's compiled and then invoked as follows:

```
java MyClass x 1
```

The output will be

```
x 1
```

We'll be getting into arrays in depth later, but for now it's enough to know that args—like all arrays—use a zero-based index. In other words, the first command-line argument is assigned to args[0], the second argument is assigned to args[1], and so on.

Example

```
1) class Abc{
    }
2) class Abc{
    void method1(){};
    }
3) class Abc{
    void method1(){
        for(int i=0;i<=5;i++){
            System.out.println(i);
        }
    }
}
```

All the above prgms will compile but not execute

- When above programs are run
 - Exception in thread "main" java.lang.NoSuchMethodError: main
- In order to execute a class it must contain main method
- A class without main method can be compiled but not executed
- **main() is the method that the JVM uses to start execution of a Java program**

```
public static void main(String[] args)
```

```
static public void main(String[] args)
```

```
public static void main(String... x)
```

```
class Abc{
```

1) Declare the class

```
    void method1(){
```

```
        for(int i=0;i<=5;i++)
```

```
        {
```

```
            System.out.println(i);
```

```
        }
```

```
    }
```

```
    public static void main(String arg[]){
```

2) PSVM

```
        Abc ob1 = new Abc();
```

3) Create instance of class

```
        ob1.method1();
```

4) Using this call methods defined on that instance

```
    }
```

```
}
```


Integer literals

decimal (base 10), octal (base 8),

hexadecimal (base 16) and binary (base 2)

numeric literals can be declared using underscore characters (_)

Decimal literals

```
int length = 343;
```

Binary Literals

```
int b1 = 0B101010;    // set b1 to binary 101010 (decimal 42)
int b2 = 0b00011;     // set b2 to binary 11 (decimal 3)
```

Octal Literals

You can have up to 21 digits in an octal number, not including the leading 0.

```
class Octal {  
    public static void main(String [] args) {  
        int six = 06;        // Equal to decimal 6  
        int seven = 07;      // Equal to decimal 7  
        int eight = 010;     // Equal to decimal 8  
        int nine = 011;      // Equal to decimal 9  
        System.out.println("Octal 010 = " + eight);  
    }  
}
```

Hexadecimal Literals

Hexadecimal (hex for short) numbers are constructed using 16 distinct symbols

0 1 2 3 4 5 6 7 8 9 a b c d e f

You are allowed up to 16 digits in a hexadecimal number, not including the prefix 0x (or 0X)

x = 1 y = 2147483647 z = -559035650

```
class HexTest {  
    public static void main (String [] args) {  
        int x = 0X0001;  
        int y = 0x7fffffff;  
        int z = 0xDeadCafe;  
        System.out.println("x = " + x + " y = " + y + " z = " + z);  
    }  
}
```

- All four integer literals (binary, octal, decimal, and hexadecimal) are defined as int by default, but they may also be specified as long by placing a suffix of L or l after the number:
- `long jo = 110599L;`
- `long so = 0xFFFFl; // Note the lowercase 'l'`

- `double d = 11301874.9881024`
- Floating-point literals are defined as double (64 bits) by default, so if you want to assign a floating point literal to a variable of type float (32 bits), you must attach the suffix F or f to the number.

```
float f = 23.467890;           // Compiler error, possible loss  
                                // of precision  
float q = 49837849.029847F;    // OK; has the suffix "F"
```

- You may also optionally attach a D or d to double literals, but it is not necessary because this is the default behaviour.

- A boolean value can be defined only as true or false.
- Although in C (and some other languages), it is common to use numbers to
- represent true or false, this will not work in Java

```
boolean t = true;    // Legal
```

```
boolean f = 0;       // Compiler error!
```

- `int x = 1; if (x) { } // Compiler error!`

- A char literal is represented by a single character in single quotes:
- `char a = 'a';`
- `char b = '@';`
- You can also type in the Unicode value of the character, using the Unicode notation of prefixing the value with `\u`, as follows:
- `char letterN = '\u004E';` // The letter 'N'
- Remember, characters are just 16-bit unsigned integers!!
- That means you can assign a number literal, assuming it will fit into the unsigned 16-bit range (0 to 65535).

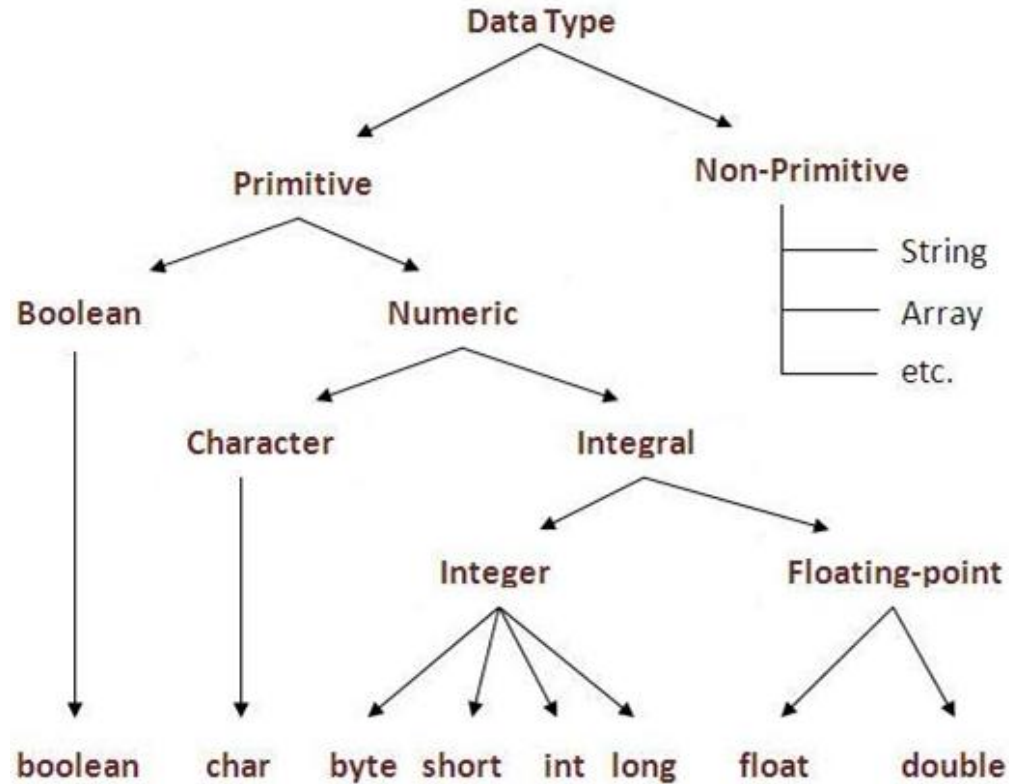
- You can also use an escape code (the backslash) if you want to represent a character that can't be typed in as a literal, including the characters for linefeed, newline, horizontal tab, backspace, and quotes:

```
char a = 0x892;           // hexadecimal literal
char b = 982;             // int literal
char c = (char)70000;      // The cast is required; 70000 is
                           // out of char range
char d = (char) -98;       // Ridiculous, but legal

char c = '\"';            // A double quote
char d = '\n';            // A newline
char tab = '\t';          // A tab
```


- A string literal is a source code representation of a value of a String object

Datatypes in Java



Datatypes in Java

Type Name	Description	Size	Range	Sample Declaration & Initialization
boolean	true or false	1 bit	{true, false}	boolean myBool = true;
char	Unicode Character	2 bytes	\u0000 to \uFFFF	char myChar = 'a';
byte	Signed Integer	1 byte	-128 to 127	int myInt = 100;
short	Signed Integer	2 bytes	-32768 to 32767	short myShort = 1000;
int	Signed Integer	4 bytes	-2147483648 to 2147483647	int myInt = 100000;
long	Signed Integer	8 bytes	-9223372036854775808 to 9223372036854775807	long myLong = 0;
float	IEEE 754 floating point	4 bytes	$\pm 1.4\text{E}-45$ to $\pm 3.4028235\text{E}+38$	float myFloat = 10.0f;
double	IEEE 754 floating point	8 bytes	$\pm 4.9\text{E}-324$ to $\pm 1.7976931348623157\text{E}+308$	double myDouble = 20.0;

Java Primitive Datatypes

- Boolean - 1-bit either true or false
- Char - 16-bit Unicode character (unsigned)
- Byte - 8-bit integer (signed)
- Short - 16-bit integer (signed)
- Int - 32-bit integer (signed)
- Long - 64-bit integer (signed)
- Float - 32-bit floating-point (IEEE 754-1985)
- Double - 64-bit floating-point (IEEE 754-1985)

We can divide all the Java operators into the following groups:

- Arithmetic Operators
- Relational Operators
- Bitwise Operators
- Logical Operators
- Assignment Operators
- Misc Operators

Arithmetic operators are

- +
- -
- *
- /
- %
- ++
- --

Relational operators

Relational operators are

- ==
- !=
- >
- <
- >=
- <=

-- Comparison gives either true or false (Ex. Comparison of 2 nos.)

Logical operators

- - &&
- - ||
- - !

Ex. Greatest of 3 numbers

Assignment operators

- =
- +=
- -=
- *=
- /=
- %=
- <<=
- >>=
- &=
- ^=
- |=

- It can be applied to the integer types, long, int, short, char, and byte
- Bitwise operator works on bits and performs bit-by-bit operation
- &
- |
- ^
- ~
- <<
- >>
- >>>
- <<<

- (?:) Conditional operator is also known as the ternary operator
- variable x = (expression) ? value if true : value if false

Example:

```
class Test {  
    public static void main(String args[]){  
        int a , b;  
        a = 10;  
        b = (a == 1) ? 20: 30;  
        System.out.println( "Value of b is : " + b );  
  
        b = (a == 10) ? 20: 30;  
        System.out.println( "Value of b is : " + b );  
    }  
}
```

- The operator checks whether the object is of a particular type(class type or interface type)
- Returns a Boolean result either true or false

Example:

```
class Test {  
    public static void main(String args[]){  
        String name = "Bunti";  
        // following will return true since name is of type String  
        boolean result = name instanceof String;  
        System.out.println( result );  
    }  
}
```

- if, multiple if, if-else, else if same as in C
- Switch statement
- It can check byte, char, int, short

Nested Switch Example:

```
switch(count){
```

```
Case 1:
```

```
    switch(target){
```

```
    case 0:
```

```
        system.out.println ("Target is zero");
```

```
        break;
```

```
    case 1:
```

```
        system.out.println("Target is one");
```

```
        break;
```

```
    }
```

```
break;
```

- while
- do-while
- for
(Variable can be declared inside for loop)
- Nested loops
- **for each loop**

For each loop

- **It is used to traverse elements of an array or collection**

Example

```
class ForEachExample1{  
    public static void main(String ar[]){  
        int arr[]={12,13,14,44};  
        for(int i:arr){  
            System.out.println(i);  
        }  
    }  
}
```

Jump Statements

- break
- continue
- return

- Used to exit from nested loops

```
outer : for(i=0;i<=3;i++){  
    for(j=0;j<=2;j++){  
        if(flag==1)  
            break outer;  
    }  
    if(flag==1)  
        break;  
}
```

return example

```
Class Test{  
    public static void main(String ar[]){  
        boolean t= true;  
        S.o.p("Before return");  
        if(t){  
            return ;  
        }  
        system.out.println("This will not be executed");  
    }  
}
```

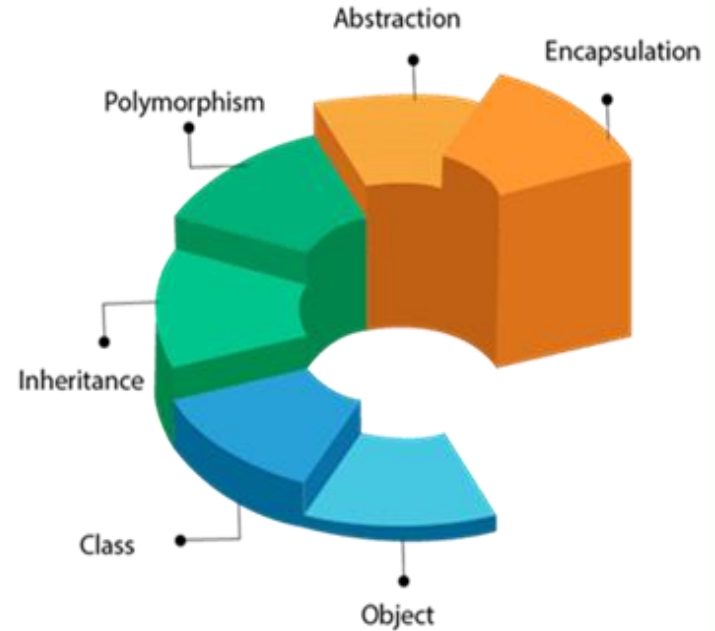
Features of Object Oriented Programming

Object Oriented Programming Concepts

- Class/Object
- Constructor
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction
- Overloading
- Overriding
- Interface

Introduction

- Object means a real-world entity such as a pen, chair, table, computer, watch, etc.
- Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects.
- It simplifies software development and maintenance by providing some concepts: Object, Class, Inheritance, Polymorphism, Abstraction, Encapsulation and Late Binding.



- Data Abstraction is the property by virtue of which only the essential details are displayed to the user.
- The trivial or the non-essentials units are not displayed to the user.
- Ex: A car is viewed as a car rather than its individual components.
- Data Abstraction may also be defined as the process of identifying only the required characteristics of an object ignoring the irrelevant details.
- The properties and behaviors of an object differentiate it from other objects of similar type and also help in classifying/grouping the objects.

- Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of car or applying brakes will stop the car but he does not know about how on pressing the accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of accelerator, brakes etc. in the car. This is what abstraction is.
- In java, abstraction is achieved by interfaces and abstract classes.
- We can achieve 100% abstraction using interfaces.

- It is defined as the wrapping up of data under a single unit.
- It is the mechanism that binds together code and the data it manipulates.
- Another way to think about encapsulation is, it is a protective shield that prevents the data from being accessed by the code outside this shield.
- Technically in encapsulation, the variables or data of a class is hidden from any other class and can be accessed only through any member function of own class in which they are declared.
- As in encapsulation, the data in a class is hidden from other classes, so it is also known as data-hiding.
- Encapsulation can be achieved by declaring all the variables in the class as private and writing public methods in the class to set and get the values of variables.

- Inheritance is an important pillar of OOP (Object Oriented Programming).
- It is the mechanism in java by which one class is allows inheriting the features (fields and methods) of another class.
- Let us discuss some of frequent used important terminologies:
- **Super Class:** The class whose features are inherited is known as superclass (or a base class or a parent class).
- **Sub Class:** The class that inherits the other class is known as subclass (or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- **Reusability:** Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class

- It refers to the ability of OOP programming languages to differentiate between entities with the same name efficiently. This is done by Java with the help of the signature and declaration of these entities.
- Polymorphism in Java are mainly of 2 types:
- Overloading
- Overriding

- **LATE BINDING:**

- In the late binding or dynamic binding, the compiler doesn't decide the method to be called.
- Overriding is a perfect example of dynamic binding.
- In overriding both parent and child classes have the same method.

- **EARLY BINDING:**

- The binding which can be resolved at compile time by the compiler is known as static or early binding. Binding of all the static, private and final methods is done at compile-time

Late Binding & Early Binding

Early Binding	Late Binding
It is a compile-time process	It is a run-time process
The method definition and method call are linked during the compile time.	The method definition and method call are linked during the run time.
Actual object is not used for binding.	Actual object is used for binding.
For example: Method overloading	For example: Method overriding
Program execution is faster	Program execution is slower

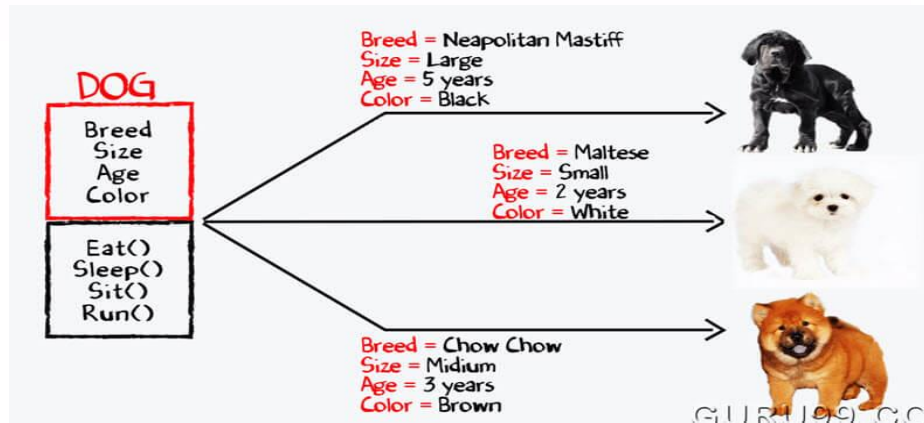
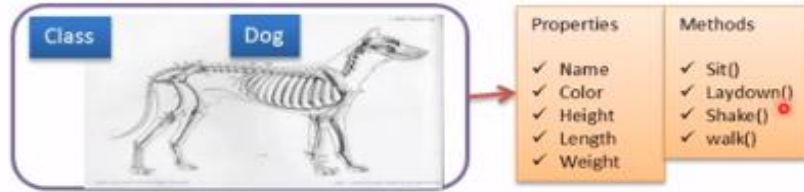
Concept of a class

They have:
ears
eyes
habitat
age



They can:
breathe()
sleep()
eat()





Concept of a class

- Classes and Objects are basic concepts of Object-Oriented Programming which revolve around the real-life entities
- Class:
 - **A class is a user defined blueprint or prototype from which objects are created.**
 - It represents the set of properties or methods that are common to all objects of one type.
 - In general, class declarations can include these components, in order:
 - Modifiers: A class can be public or has default access
 - class keyword: class keyword is used to create a class.
 - Class name: The name should begin with an initial letter (capitalized by convention).

Class – Idea/blueprint

Object – Implementation of idea/implementation of blueprint

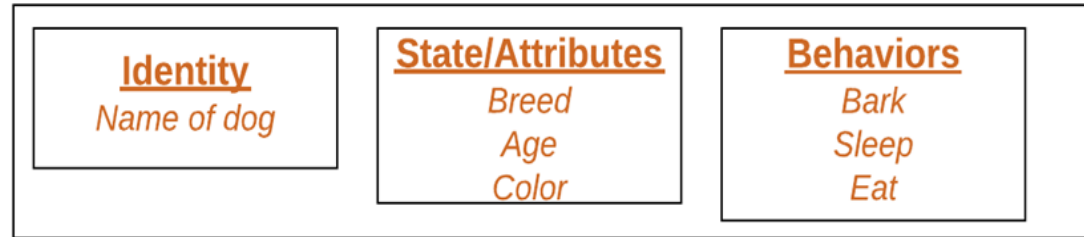
- Class:
 - Superclass (if any): The name of the class's parent (superclass), if any, preceded by the keyword extends. A class can only extend (subclass) one parent.
 - Interfaces (if any): A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword implements. A class can implement more than one interface.
 - Body: The class body surrounded by braces, { }.
 - Constructors are used for initializing new objects.
 - Fields are variables that provide the state of the class and its objects, and methods are used to implement the behavior of the class and its objects

Concept of an object

- It is a basic unit of Object-Oriented Programming and represents the real life entities. A typical Java program creates many objects, which as you know, interact by invoking methods.
- An object consists of:
 - State: It is represented by attributes of an object. It also reflects the properties of an object.
 - Behavior: It is represented by methods of an object. It also reflects the response of an object with other objects.
 - Identity: It gives a unique name to an object and enables one object to interact with other objects.
- Example of an object: dog

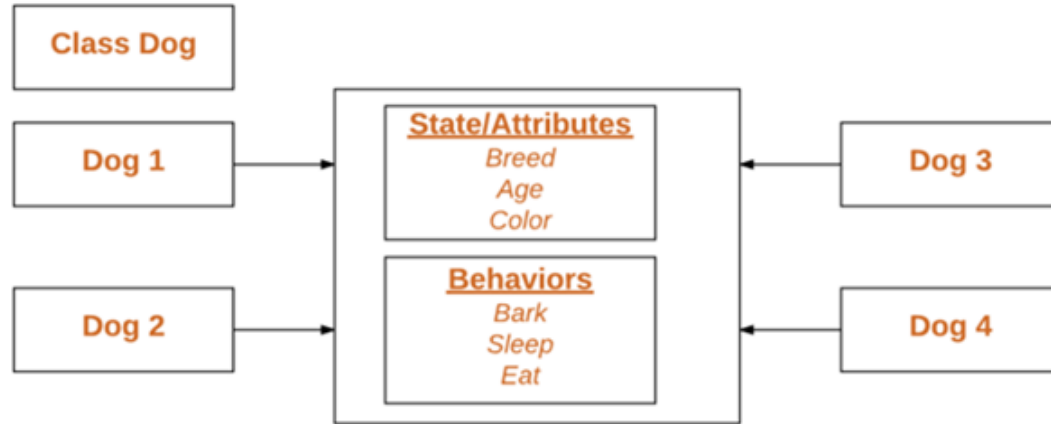
Class – Idea/blueprint

**Object – Implementation of
idea/implementation of blueprint**



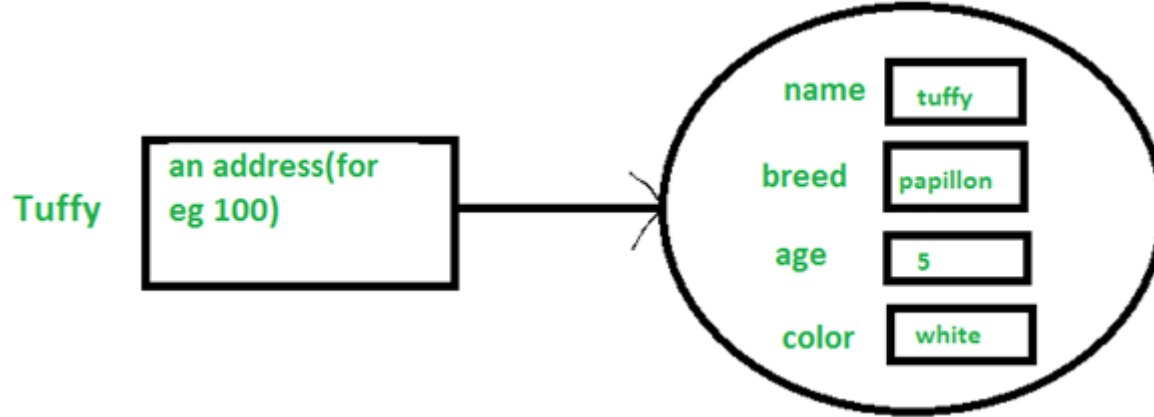
Instantiating a class

- Example:



Initializing an object

- The new operator instantiates a class by allocating memory for a new object and returning a reference to that memory. The new operator also invokes the class constructor.
- `Dog tuffy = new Dog("tuffy","papillon",5, "white");`



- Ex.1
- **Animal** class – color, size,habitat, age, etc.
- Operations – eat(), sleep(),die(), etc.
- Ex. 2
- **Employee** class – name, age,bs, etc.
- Operations – workOverTime(), takeLeave(), payTax(), etc.

Difference between class and object

- Class is **static view** of the system whereas Object is **dynamic view** of the system
- **Interaction** among Class is **not possible** whereas Interaction among **objects** is **possible** (message passing; calling methods)

Coding convention for Class

A class should have :

- Set of attributes (variables)
- Setter (Change) methods – set attribute values
- Getter methods – fetch attribute values
- Constructor
- Set of other methods

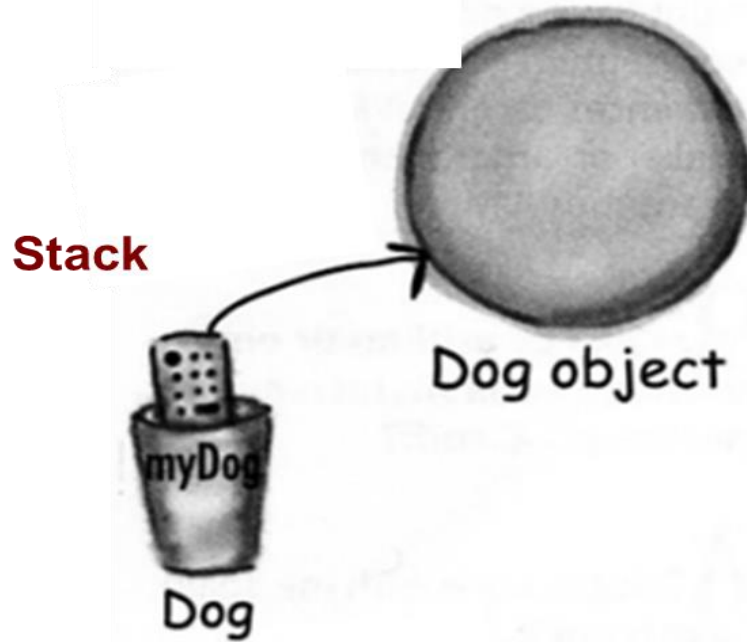
Creating an object (**Instantiating a class**)

1. Declare a reference variable
2. Create an object using **new**
(new tells JVM to allocate space for a new object on the heap)
3. Link the object and reference
4. Use dot (.) operator to access members of object

NOTE

- When an object of a class is created, the class is said to be instantiated.
- All the instances share the attributes and the behavior of the class.
- But the values of those attributes, i.e., the state are unique for each object.
- A single class may have any number of instances.

Reference and object



`Dog myDog = new Dog();`

More on reference

1. Size of object reference is JVM dependent, mostly 64 bits
 2. On a single JVM, size of all object references is same
 3. Arithmetic operations cannot be performed on object references
-
1. Object references can be set to null

Class & Objects - Example

```
class Student {  
    String name;  
    int rollno;  
}  
class ClassDemo {  
    public static void main(String args[]){  
        Student s1; // Class Variable creation : Acts as a reference to an object  
        s1 = new Student(); //New operator creates actual object whose address is stored in s1 , that is a  
            reference variable  
        s1.name = "Raj Kumar";  
        s1.rollno = 3;  
        System.out.println("Name is: "+s1.name+" Roll No. is : "+s1.rollno);  
    }  
}
```

Copying one reference into another

Example::

- `Book b= new Book();`
- `Book c = new Book();`
- `Book d=c;`
- Here, c and d act as 2 remote controls for same TV
- `b=c;`
- Now, object 1 is abandoned and is eligible for garbage collection
- `c=null; // does not refer to anything`

Comparing reference variables : DO IT YOURSELF

- In above example,
if(b == c){
}

Above comparison will result into ??

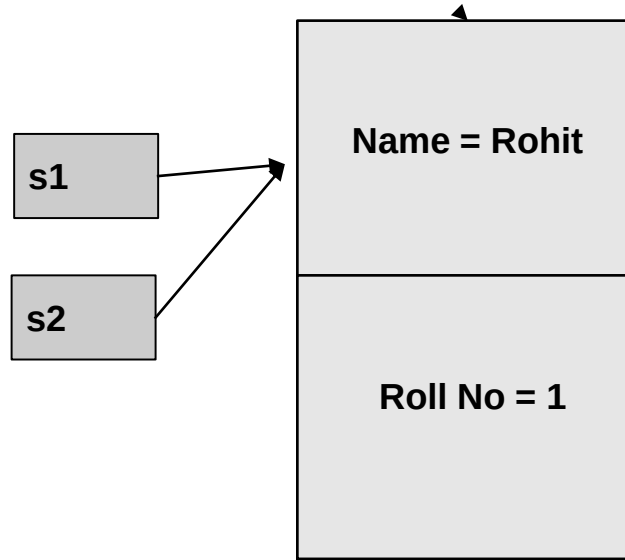
Copying one reference into another

```
> class Student {  
    name  
    roll no  
}
```

```
Student s1 = new Student("Rahul",1)
```

```
Student s2 = s1;
```

- > s1 and s2 refer to same m/m location
- > s1 = null will not change s2



Class name & File name ??

```
Class A{  
}
```

```
Class B{  
}
```

Saved as Test.java

On compilation it forms two class files viz. A.class and B.class

java A or java B or java Test – Error

Searches main method in the name of the class passed as an argument

```
class A {  
    public static void main(String a[]){  
    }  
}  
class B {  
    public static void main(String a[]){  
    }  
}
```

This code compiles successfully

Command --> java A OR java B works

On compilation A.class and B.class are created. On execution JVM searches for main method in the argument file passed in cmd “java filename.class”

- There is no restriction that class name should match file name unless class is public
- Number of class files formed is equal to no. of classes in java program
- Only if a class is public then file name and class name should match

Try yourself :Find the output

```
class A {  
    void pqr(){  
  
        System.out.println("Inside pqr of  
A");  
    }  
    public static void main(String ar[] {  
  
        System.out.println("From A");  
        B b = new B();  
        b.xyz();  
  
    }  
}
```

```
class B {  
  
    void xyz(){  
  
        System.out.println("Inside xyz of B");  
    }  
    public static void main(String ar[] {  
        System.out.println("From B");  
        A a = new A();  
        a.pqr();  
    }  
}  
Saved as "Ab.java"
```

Concept of Constructor

- Used to initialize member variables of a class during object creation
- **A new object can't be created without invoking a constructor**
- Invoked automatically when 'new' keyword encountered
- Has same name as that of class
- No return type; not even void
- **Every class must have a constructor**
- If no constructor is defined, a **default constructor** (no-argument constructor) is automatically generated by compiler
- Constructors can have **access modifiers**
- Constructors look a little strange because they have no return type, not even void. This is because the implicit return type of a class' constructor is the class type itself
- It is the constructor's job to initialize the internal state of an object so that the code creating an instance will have a fully initialized, usable object immediately

CONSTRUCTOR : Example

```
class Box {  
    double width;  
    double height;  
    double depth;  
    // This is the constructor for Box.  
    Box() {  
        System.out.println("Constructing Box");  
        width = 10;  
        height = 10;  
        depth = 10;  
    }  
    // compute and return volume  
    double getVolume() {  
        return width * height * depth;  
    }  
}
```

```
class BoxDemo6 {  
    public static void main(String args[]) {  
        // declare, allocate, and initialize Box  
        // objects  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
        double vol;  
        // get volume of first box  
        vol = mybox1.getVolume();  
        System.out.println("Volume is " + vol);  
        // get volume of second box  
        vol = mybox2.getVolume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

OUTPUT:

```
Constructing Box  
Constructing Box  
Volume is 1000.0  
Volume is 1000.0
```

CONSTRUCTOR : Example Discussion

- As you can see, both mybox1 and mybox2 were initialized by the Box() constructor when they were created.
- Since the constructor gives all boxes the same dimensions, 10 by 10 by 10, both mybox1 and mybox2 will have the same volume.
- The println() statement inside Box() is for the sake of illustration only. Most constructors will not display anything. They will simply initialize an object.
- Before moving on, let's reexamine the new operator.
- As you know, when you allocate an object, you use the following general form:
 class-var = new classname();
- Now you can understand why the parentheses are needed after the class name. What is actually happening is that the constructor for the class is being called.
- Thus, in the line Box mybox1 = new Box(); *new Box()* is calling the Box() constructor

- When you do not explicitly define a constructor for a class, then Java creates a default constructor for the class. This is why the earlier versions of code worked, that did not define a constructor.
- The default constructor automatically initializes all instance variables to their default values, which are zero, null, and false, for numeric types, reference types, and boolean, respectively.
- The default constructor is often sufficient for simple classes, but it usually won't do for more sophisticated ones. Once you define your own constructor, the default constructor is no longer used

- While the Box() constructor in the preceding example does initialize a Box object, it is not very useful—all boxes have the same dimensions. What is needed is a way to construct Box objects of various dimensions.
- The easy solution is to add parameters to the constructor. As you can probably guess, this makes it much more useful.
- For example, the following version of Box defines a parameterized constructor that sets the dimensions of a box as specified by those parameters. Pay special attention to how Box objects are created

Parameterized Constructor : Example

```
class Box {  
    double width;  
    double height;  
    double depth;  
    // This is the constructor for Box.  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
    // compute and return volume  
    double getVolume() {  
        return width * height * depth;  
    }  
}
```

```
class BoxDemo7 {  
    public static void main(String args[]) {  
        // declare, allocate, and initialize Box  
        // objects  
        Box mybox1 = new Box(10, 20, 15);  
        Box mybox2 = new Box(3, 6, 9);  
        double vol;  
        // get volume of first box  
        vol = mybox1.getVolume();  
        System.out.println("Volume is " + vol);  
        // get volume of second box  
        vol = mybox2.getVolume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

OUTPUT:
Volume is 3000.0
Volume is 162.0

Parameterized Constructor: Example Discussion

- As you can see, each object is initialized as specified in the parameters to its constructor.
- For example, in the following line,
`Box mybox1 = new Box(10, 20, 15);`
- the values 10, 20, and 15 are passed to the `Box( )` constructor when `new` creates the object.
- Thus, `mybox1`'s copy of width, height, and depth will contain the values 10, 20, and 15, respectively

Find the output

```
class Foo{  
    String name;  
    int age;  
    Foo(String n, int a){  
        name=n;  
        age=a;  
    }  
}  
  
Foo f=new Foo();  
Foo f=new Foo("Foo1",100);
```

Home Work

- Can a class have a method with same name as that of class??
- Can we explicitly call a constructor??
- What is the access modifier of a default constructor??

Will compiler insert default constructor??

```
class Horse {  
    Horse() { }  
    Horse(String name) { }  
}
```

Will the compiler put in a default constructor for this class? ☐

How about for the following variation of the class?

```
class Horse {  
    Horse(String name) { }  
}
```

Now will the compiler insert a default constructor? ☐

What about this class?

```
class Horse { }
```

Okay, what about this class?

```
class Horse {  
    void Horse() { }
```

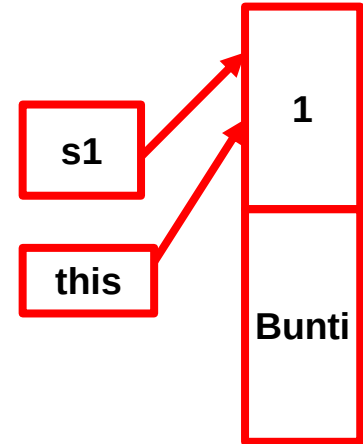
'this' keyword

- It is a reference variable that refers to **current (invoking) object in a method**

Example

```
class Student{
    int rollNo;
    String name;
    Student(int rollNo,String name){
        this.rollNo=rollNo;
        this.name=name;
    }
    void display(){
        System.out.println("Roll No. =" + this.rollNo);
        System.out.println("Name    =" + this.name);
    }
}

class ThisDemo{
    public static void main(String ar[]){
        Student s1 = new Student(1,"Bunti");
    }
}
```



'this' Keyword: Example

```
Box(double w, double h, double d) {  
    this.width = w;  
    this.height = h;  
    this.depth = d;  
}
```

- This version of Box() operates exactly like the earlier version. The use of this is redundant, but perfectly correct.
- Inside Box(), this will always refer to the invoking object. While it is redundant in this case, this is useful in other contexts

- To **refer to current class instance variable** or data member
- **Resolves ambiguity** between instance variable and formal parameter

Example

```
Student (String Name,int rollNo){  
    name=name;  
    rollNo=rollNo;  
}  
PSVM(){  
    Student s1=new Student(“Rajkumar”,10);  
}
```


Usage 2

- To invoke current class constructor
- Used for constructor chaining
- *'this'* should be the first statement in constructor

Example

```

Class Student{
    String name; int rollNo;
    Student(){
    }
    Student(int r){
    }
    Student (String n, int r){
        this(r);
        name= n;
    }
}

Student s= new Student("XYZ",1);
    
```

Usage 3

- Used to invoke current class method
- Used for method chaining

Example

- Define a class Test with 3 method p(), m(), n()
- Calling sequence
 - $\text{main()} \rightarrow \text{p()} \rightarrow \text{m()} \rightarrow \text{n()}$
 - Use 'this' keyword

```
Class Test{  
    void n() {  
        SOP("n is called");  
    }  
    void m(){  
        this.n();  
    }  
    void p() {  
        this.m();  
    }  
}
```

```
Test t1 = new Test();  
t1.p();
```

- **'this' can be passed as a parameter in a method call or constructor call**

Example – application (DO IT YOURSELF)

- Define class Test with
 - `int x;`
 - `method1()` should call `method2()`
 - `method2(Test ob)` should print the value of `x`
 - Create an object of Test and appropriately call methods of Test class

Question:

Define a class employee with name, age and bs.

Define a constructor to initialize the data members

Define set and get methods for data members

Define a method to calculate gross salary of employee where $gs = da + hra + bs$. $Da = 10\%$ of bs and $HRA = 20\%$ of bs

END OF UNIT 1